

TRIVUS

Plataforma do Ecossistema Trivus
Documentação Completa do Sistema

Visão de negócio · Regras e nuances · Backend · Banco de dados ·
Frontend (futuro) · Infra e Roadmap

Julho de 2026 · v1.0

Backend: github.com/trivusauto/trivus-api

1. O que é o sistema

A **Trivus** é uma holding que presta serviços completos para concessionárias e revendas de veículos no Brasil: **consultoria de gestão, capacitação de equipes e agência de marketing**. Este sistema é a **plataforma de software desse ecossistema** — o produto que conecta os serviços da holding às operações diárias das lojas.

Na prática, o sistema é um **CRM + central de métricas para lojas de veículos**, com captação automática de leads pelo WhatsApp, gestão de agendamentos e vendas, análise de marketing por custo de funil e por campanha, metas e planos de ação — e, por cima de tudo, uma camada de **gestão comercial da própria holding**: empresas, planos, assinaturas (SaaS ou consultoria completa), serviços contratáveis e venda de upgrade (upsell) dentro do próprio produto.

1.1 Os dois modos de uso

Modo	Como funciona
Consultoria completa	A empresa é cliente da holding: usa o software inteiro, com acompanhamento humano (consultoria, capacitação, agência). Cobrança por contrato, gerida manualmente.
SaaS	A empresa assina só o software (ou parte dele). Pode começar com trial, usar módulos específicos e evoluir para a consultoria completa apenas trocando o plano — sem migração de dados ou de sistema.

A regra de desenho central: **migrar entre os modos é trocar o plano da assinatura**. Os dados, as lojas e os usuários permanecem intactos.

1.2 Histórico e contexto técnico

O sistema original era um frontend Next.js que falava diretamente com o Supabase (banco exposto ao navegador, regras de negócio no cliente, sem camada de API). Este projeto substitui essa arquitetura por um **backend próprio (FastAPI)** com banco remodelado, para permitir escala, segurança e a reescrita do frontend sem acoplamento — a camada de API é o que protege o negócio de futuras migrações.

2. Domínio: conceitos e papéis

2.1 A hierarquia do ecossistema

Conceito	O que é
Empresa (company)	A conta comercial — quem assina o contrato com a holding. Uma empresa pode ter várias lojas.
Loja (store)	O tenant operacional: onde vivem os leads, funis, métricas e a equipe.
Assinatura (subscription)	O contrato da empresa: plano + status (trialing / active / suspended / canceled) + forma de cobrança.
Plano (plan)	Combinação de serviços que a empresa PODE usar (ex.: SaaS Starter, SaaS Pro, Consultoria Completa) + limite de lojas.
Serviço (service)	Item do catálogo do ecossistema, gerido pelo admin. Pode ser de software (desbloqueia telas/recursos) ou humano (consultoria, capacitação, agência — aparece no catálogo/upsell).
Feature key	Chave técnica de um recurso do software (tela, card ou área de tela). Os serviços apontam para as keys que desbloqueiam. Ex.: metrics.reports.costs = só a coluna de custos do relatório.

2.2 Papéis de usuário

Papel	Quem é	O que enxerga
admin	Equipe da holding Trivus	Tudo, todas as lojas. Nunca é bloqueado por assinatura/serviço.
client	Dono de loja(s)	Suas lojas (vínculo N:N). É o 'gestor': vê todos os leads e métricas das lojas dele.
shop_user	Colaborador de 1 loja	Conforme o shop_role: gerente (tudo da loja), vendedor, sdr, administrativo. SDR pode ter a permissão 'ver leads não atribuídos', que o coloca no rodízio automático do WhatsApp.

2.3 Glossário do dia a dia

Termo	Significado
funil (campo do lead)	A ORIGEM do lead: 'receptivo' (chegou sozinho: WhatsApp, indicação), 'prospeccao_ativa' (a equipe foi atrás), qualquer outro valor = 'outros'. Vazio = receptivo. NÃO confundir com o funil-Kanban.
Funil (Kanban)	O pipeline de etapas por onde o lead avança. Cada loja recebe um CLONE do funil-template criado pelo admin; mudanças no template são propagadas para os clones.
lid	Identificador único do chat de WhatsApp (Z-API). Usado na deduplicação de leads.
rentabilidade	Margem da venda: receita - custo do veículo.
assigned_to / vendedor_id / agendado_por	SDR responsável pelo 1º contato / vendedor do atendimento presencial / quem marcou o agendamento.
cooling	Alerta visual de lead 'esfriando' quando fica X horas parado numa etapa.
Modo indicadores	Alternativa ao CRM: a loja lança números diários (leads, agendamentos, vendas, investimento...) num formulário, sem mover cards.

3. Regras de negócio e nuances (o coração do sistema)

3.1 O ciclo de vida do lead

O Kanban padrão tem 7 etapas. Para **avancar** um lead, os campos obrigatórios da etapa de destino (e de todas as intermediárias, se pular etapas) precisam estar preenchidos. Voltar etapa não valida nada.

Etapa	Campos obrigatórios para entrar
1. RECEBIDOS	funil (origem) e telefone
2. CLASSIFICADOS	nome e cidade (= o SDR fez contato)
3. QUALIFICADOS	modelo do veículo e ano (= interesse real)
4. AGENDADOS	data e horário do agendamento
5. EM ATENDIMENTO	compareceu = sim e, depois, vendedor responsável
6. VEÍCULOS COMPRADOS	valor de compra (veículo que a loja comprou do cliente na troca)
7. VEÍCULOS VENDIDOS	receita, despesa e rentabilidade (negócio fechado)

Nuances importantes:

- Nomes de etapa são normalizados (sem acento, maiúsculas) — renomear cosmético não quebra as regras.
- Datas automáticas (o sistema seta sozinho, o usuário não digita): 1º agendamento completo grava `agendado_por` = usuário e `data_marcacao_agendamento` = hoje; 1º 'compareceu' grava `data_compareceu` = hoje; fechamento grava `data_fechou_negocio` = hoje. Desmarcar limpa os campos correspondentes.
- Dinheiro aceita formato brasileiro ('1.000,00') nos fechamentos.
- Todo movimento de etapa grava histórico (com timestamp) e log de atividade — o histórico alimenta as métricas.
- Bloqueio por campanha: se a loja ligar a flag 'exigir campanha', lead RECEPTIVO sem campanha vinculada não avança de etapa.

3.2 'Qual data conta' — a regra de ouro das métricas

Cada métrica conta na data do SEU evento, em **horário de Brasília** (nunca UTC):

Métrica	Conta na data de...	Detalhe
Leads	<code>created_at</code> (entrada no sistema)	convertido para data local
Agendamentos	<code>data_marcacao_agendamento</code>	quando foi MARCADO, não a data do compromisso; fallback: <code>data_agendamento</code>
Comparecimentos	<code>data_compareceu</code>	com <code>compareceu</code> = sim
Conversões / Receita	<code>data_fechou_negocio</code>	receita = soma das rentabilidades
Qualificados / Classificados	<code>created_at</code> do lead	lead conta se ENTROU na etapa (histórico) OU está nela/posterior — mesmo que tenha regredido depois

3.3 Dias úteis (projeções)

Projeção do mês = realizado + (ritmo por dia útil x dias úteis restantes). **Sábado conta** como dia útil; domingo não; feriados nacionais brasileiros não — incluindo os móveis calculados pela Páscoa (Carnaval):

-48/-47 dias; Sexta-feira Santa: -2; Corpus Christi: +60).

3.4 Captação automática pelo WhatsApp (Z-API)

Cada loja tem um token único de webhook. Quando chega mensagem no WhatsApp da loja:

- Ignora: grupos, mensagens enviadas pela própria loja (fromMe), newsletters/canais.
- Extrai o telefone (ou o lid, quando o número é anônimo) e normaliza o formato brasileiro.
- **Deduplicação:** procura lead existente pelo lid e pelas variantes do telefone com/sem o 9º dígito ((11) 99999-9999 e (11) 9999-9999 são o mesmo número). Se existe, enriquece o cadastro e não duplica.
- Cria o lead na 1ª etapa (RECEBIDOS) do funil clonado, com origem 'receptivo'.
- **Rodízio (round-robin):** atribui ao próximo SDR com permissão 'ver leads não atribuídos', em fila circular; o ponteiro do último SDR fica salvo NA LOJA.
- **Campanha automática:** tenta identificar a campanha de origem procurando o link_code das campanhas ativas no payload (link do anúncio/texto). O preenchimento manual é o fallback garantido.

3.5 Marketing: o funil de custos

O módulo de marketing analisa apenas o funil **receptivo** (prospecção ativa não tem investimento de mídia — é custo de mão de obra). Para o período selecionado:

Etapa do funil	Quantidade	Custo unitário
Leads	leads receptivos criados	investimento / leads = CPL
Classificados	que chegaram a CLASSIFICADOS+	investimento / classificados
Qualificados	que chegaram a QUALIFICADOS+	investimento / qualificados
Agendados	agendamentos marcados	investimento / agendados
Comparecidos	comparecimentos	investimento / comparecidos
Vendas	negócios fechados	investimento / vendas = CAC

- Cada etapa exibe também a taxa de conversão da etapa anterior (ex.: 80% dos leads viraram classificados).
- **ROAS** = faturamento / investimento · **ROI** = (faturamento - investimento) / investimento.
- **Sinaleiro** por etapa: verde $\geq 100\%$ da meta, amarelo $\geq 80\%$, vermelho abaixo, cinza sem meta. Metas são mensais por loja — consulta multi-loja ou multi-mês fica cinza.
- **Investimento realizado** = soma dos lançamentos diários de investimento em marketing (modo indicadores). **Funil por campanha** usa o budget da campanha como investimento (CAC da campanha = budget / vendas da campanha).
- Relatórios ganham o filtro por campanha: selecionando uma campanha, todo o funil mostra só os leads dela.

3.6 Modo indicadores (lojas sem CRM ativo)

- Lançamento diário por (loja + data + origem) com UPSERT — reenviar o mesmo dia atualiza, nunca duplica.
- Campos: totais de leads, classificados, qualificados, agendamentos, comparecimentos, conversões, rentabilidade bruta, despesas do dia, investimento em marketing e notas.
- Regras herdadas do sistema antigo (preservadas à risca): total/classificados/qualificados contam SÓ receptivo; receita exibida é LÍQUIDA (rentabilidade - despesas diárias).
- O relatório compara realizado x metas por origem, incluindo a meta de investimento em marketing.

3.7 Ecossistema: assinaturas, serviços e upsell

A cadeia de acesso (avaliada a cada requisição): um recurso está liberado para uma loja quando: (1) a assinatura da empresa está utilizável — active, ou trialing dentro do prazo; (2) o plano contém o serviço; (3) o serviço está LIGADO naquela loja (modelo híbrido: contrata-se por empresa, liga-se por loja); e (4) o recurso pertence às feature keys do serviço.

- **Trial expira sozinho**: trialing com data vencida é tratado como suspenso na hora, sem rotinas agendadas.
- **Admin nunca é bloqueado**: loja ainda sem empresa vinculada opera em 'modo legado' (sem bloqueios) até a migração de dados — ninguém perde acesso durante a transição.
- **Upsell**: quando o usuário encontra um recurso bloqueado, o sistema mostra o card do serviço que o desbloqueia (nome, o que é, pitch). O clique em 'quero conhecer' registra o interesse (fila comercial: novo -> contatado -> convertido/descartado) e dispara notificação automática ao comercial da holding

via n8n.

- **Catálogo com CRUD:** o admin cria/edita serviços e monta o que cada um desbloqueia escolhendo as feature keys numa lista fechada (definida no código — sem digitação livre, sem risco de typo derrubar acesso). A key do serviço é imutável e a exclusão é bloqueada se o serviço estiver em algum plano.

3.8 Cobrança

- v1: cobrança MANUAL (admin cria/ativa/suspende assinaturas) + trial automático.
- A integração com o framework de pagamentos próprio da holding (que já fala com vários gateways) está DESENVOLVIDA e DESLIGADA por flag: o framework chamará um endpoint de eventos (pagamento confirmado / falhou / vencido / estornado); o sistema registra tudo em histórico e, para assinaturas em modo 'gateway', ativa/suspende sozinho. Guia completo: docs/BILLING_GATEWAY.md.

3.9 Disparos em massa (WhatsApp)

- Campanhas de mensagens para listas de contatos: telefones são validados, normalizados e deduplicados no servidor.
- Até 5 variações de mensagem (distribuídas em rodízio) e atraso aleatório entre envios (min/max).
- O processamento é do n8n, que devolve o status de cada contato (pendente / enviado / falhou) por endpoint autenticado.

4. O Backend (implementado)

Repositório: github.com/trivusauto/trivus-api · Estado: **pronto** — 154 testes automatizados verdes, tipagem estrita (mypy) e lint (ruff) limpos, CI no GitHub Actions, deploy automático via Coolify.

4.1 Stack

Camada	Tecnologia
Linguagem / Framework	Python 3.12 · FastAPI (OpenAPI/Swagger automáticos em /docs)
Banco / ORM / Migrations	PostgreSQL 16 · SQLAlchemy 2 async · Alembic (3 migrations, 23 tabelas)
Auth	JWT (Bearer) + argon2; senhas do sistema antigo são aceitas e re-hasheadas no 1º login
Qualidade	pytest (unit + integração + e2e) · ruff · mypy strict · TDD
Infra	Docker/Compose · GitHub Actions (CI + deploy) · Coolify

4.2 Arquitetura hexagonal (DDD)

Cada módulo em `src/modules/<nome>` tem 4 camadas com dependências apontando para dentro:

- **domain/** — entidades e serviços de regra PUROS (regras de etapa, patches do lead, métricas, dias úteis, funil de custos, telefone/rodízio, resolução de acesso). Zero I/O; 100% testados unitariamente.
- **application/** — use cases (1 classe = 1 caso de uso com `execute()`), testados com fakes.
- **infrastructure/** — SQLAlchemy (ORM + repositórios), clientes externos (n8n), leitores de consulta.
- **interface/** — routers FastAPI, schemas Pydantic e injeção de dependências.

4.3 Módulos e endpoints (~80 rotas)

Módulo	Responsabilidade	Principais rotas
auth	login, sessão, senha, RBAC, escopo multi-loja	POST /auth/login · GET /auth/me · POST /auth/change-password
stores + users	lojas, usuários portal, colaboradores, vínculos, rótulos de papéis	/admin/stores · /admin/users · /stores/{id}/team · role-labels
crm	Kanban completo + templates do admin	/crm/funnels · /crm/leads (+stage, /agendamento, /comparecimento, /fechamento) · /crm/stages (+cooling) · /admin/crm/templates (+sync)
webhook	captação WhatsApp	POST /webhook/zapi/{token} (público, por token de loja)
agenda	agendamentos com filtros e escopo	GET /agenda (presets, busca, paginação 25/50/100)
metrics	dashboards e análises	/metrics/dashboard · /reports · /projections · /team · /indicators-report
marketing	campanhas e funil de custos	/campaigns · /marketing/funnel · /marketing/by-campaign
indicators/goals/action_plans/legacy	modo indicadores, metas, planos de ação	/indicators (upsert) · /goals + /admin/goals (upsert) · /action-plans (+status) · /leads (legado)
bulk_send	disparos em massa	/admin/bulk-sends (+logs) · /integrations/bulk-send/... (n8n)

Módulo	Responsabilidade	Principais rotas
ecosystem	empresas, planos, assinaturas, serviços, gates, upsell, cobrança	/ecosystem/my-entitlements · /ecosystem/services · /ecosystem/interests · /admin/{services,plans,companies,subscriptions,interests} · /integrations/billing/events

Referência completa endpoint a endpoint (entrada/saída/permissões/regras):

docs/API_REFERENCE.md no repositório — escrita para que uma IA construa o frontend sem precisar ler o código. Schemas exatos: GET /openapi.json.

4.4 Convenções da API

- Erros: {"error": "mensagem em português"} com status 400/401/403/404; validação de payload em 422.
- Recurso bloqueado por assinatura: 403 {"error": "feature_locked", "feature_key": "..."} — contrato que o frontend usa para renderizar o card de upsell no lugar exato do conteúdo.
- Datas YYYY-MM-DD; dinheiro em float (2 casas); IDs UUID; consultas sempre escopadas por store_id.

5. Banco de dados (23 tabelas)

O banco foi **remodelado do zero** (o antigo não tinha chaves estrangeiras e misturava conceitos).
Decisões estruturais:

- **FKs reais e enforçadas** em todos os relacionamentos (o antigo usava convenção de nomes).
- **users** guarda só credenciais/perfil; dados de loja vivem só em **stores** (fim da duplicação legada).
- `client_id` -> `store_id` em todas as tabelas de negócio; `parent_client_id` -> `parent_store_id`.
- Tipos corretos (`date/timestamptz` para datas, `numeric(14,2)` para dinheiro), **CHECKs** para enums, índices nas colunas de consulta pesada, `soft-delete` via `'active'`.
- Migrations versionadas (Alembic) — o schema é reproduzível do zero em qualquer ambiente.

Grupo	Tabelas	Observações
Acesso	users, stores, user_store_access	vínculo N:N com <code>is_owner</code> ; stores carrega as flags operacionais (<code>crm_enabled</code> , <code>webhook</code> , <code>exigir campanha</code>) e <code>company_id</code>
CRM	crm_funnels, crm_funnel_stages, crm_funnel_leads, crm_lead_stage_history, crm_stage_cooling_rules, crm_activity_log	funis-template (<code>store_id</code> nulo) e clones (<code>template_source_id</code>); lead com ~35 campos incluindo <code>campaign_id</code>
Negócio	leads (legado), daily_indicators, goals, action_plans	uniques que viram upsert: (loja,data,origem) e (loja,ano,mês,origem)
Marketing	marketing_campaigns	budget, período, <code>link_code</code> p/ auto-match no WhatsApp
Disparos	bulk_sends, bulk_send_contacts	até 5 variações; status por contato
Ecossistema	companies, plans, subscriptions, services, store_services, service_interests, subscription_payments	planos e serviços guardam listas de keys em JSONB; pagamentos guardam o payload bruto do framework p/ auditoria

DDL completo e comentado: **docs/db/MODELO_ALVO.md** no repositório.

5.1 Migração dos dados do sistema antigo (pendente)

Os dados de produção continuam no Supabase até o corte. O plano de migração (Plano 11) já está escrito: script ETL idempotente com o de-para de colunas, **ensaiado contra uma cópia do banco real** até as contagens baterem; virada única em janela curta de manutenção; o banco antigo permanece intacto como rollback. Nesse mesmo passo, cada cliente atual vira uma empresa com assinatura 'Consultoria Completa' ativa — ninguém percebe a mudança.

6. O Frontend (a ser implementado)

O frontend atual (Next.js falando direto com o Supabase) será **reescrito** consumindo exclusivamente a API. Diretrizes já definidas:

6.1 Stack e arquitetura recomendadas

- Next.js (App Router) + **TypeScript** + Tailwind — mantém o que já funciona bem no visual atual.
- **TanStack Query** para estado de servidor (cache, loading, invalidação); contexts/Zustand só para estado de UI (tema, loja selecionada, modais).
- Cliente HTTP único e tipado, com tipos **gerados do OpenAPI** do backend (openapi-typescript) — segurança de tipos de ponta a ponta sem duplicação.

- Organização por feature (features/crm, features/metrics...), com hooks de dados por módulo.
- JWT em cookie httpOnly + middleware de rota; o RBAC visual usa o papel do usuário, mas a segurança real é do backend.

6.2 O contrato de entitlements (a novidade que o front precisa dominar)

- Ao trocar de loja: GET /ecosystem/my-entitlements -> lista de feature keys -> o front esconde menus, telas e cards que não estão na lista.
- Recurso bloqueado (ou resposta 403 feature_locked): renderizar no lugar o card do serviço que desbloqueia (dados vêm de GET /ecosystem/services, campo locked_unlockers) com CTA 'quero conhecer' -> POST /ecosystem/interests.
- Áreas parciais: a API já corta os dados (ex.: 'costs' vem nulo sem a key da coluna de custos) — o front só decide o que mostrar no lugar.

6.3 Telas por módulo (mapa do que será construído)

Área	Telas
Autenticação	Login · troca de senha · seletor de loja (multi-loja)
CRM	Kanban (colunas, drag-and-drop com validação de campos, cooling visual, modal do lead com patches de agendamento/comparecimento/fechamento) · histórico/atividades
Agenda	Lista e calendário (dia/semana/mês), filtros por tipo de data e período, atribuição de vendedor
Métricas	Dashboard (cards + série 12 meses) · Relatórios por origem com coluna de custos e filtro por campanha · Projeções (trio Meta/Realizado/Projetando + sinaleiros) · Performance da equipe
Marketing (novo)	Seção 1: funil de custos geral (CPL...CAC, ROAS/ROI, sinaleiros) · Seção 2: um funil por campanha · Seção 3: gráfico comparativo entre campanhas · cadastro de campanhas
Indicadores / Metas / Planos	Lançamento diário (upsert) · comparação com metas · planos de ação com status
Admin holding	Lojas e usuários · funis-template (+sync) · disparos em massa · empresas, planos, assinaturas, catálogo de serviços (picklist de feature keys), fila de interesses (upsell)

6.4 Estratégia de entrega

Reescrita tela a tela (Strangler Fig): o front novo entra módulo a módulo consumindo a API, começando pelo CRM. O documento docs/API_REFERENCE.md + /openapi.json são o insumo completo para uma IA gerar essas telas com contratos corretos.

7. Infraestrutura, deploy e operação

- **CI (GitHub Actions):** a cada push — lint (ruff) + tipos (mypy strict) + 154 testes contra Postgres real.
- **Deploy (Coolify):** com o CI verde na main, o pipeline dispara o webhook do Coolify, que recria pelo Dockerfile; as migrations rodam automaticamente no boot do container. Guia: docs/DEPLOY_COOLIFY.md.
- **Ambientes/secrets:** DATABASE_URL (asyncpg), JWT_SECRET, tokens do n8n (disparos + upsell) e da cobrança, flag BILLING_GATEWAY_ENABLED. Todos documentados no README.
- **Observação de segurança:** as chaves do Supabase antigas ficaram expostas no projeto legado e devem ser rotacionadas no corte; a senha padrão do admin seedado deve ser trocada no 1º acesso em produção.

8. Roadmap e pendências

Quando	Entrega	Status
Julho/2026	Backend completo + módulo Marketing (planos 02-10b, 12)	CONCLUÍDO — 154 testes, deploy automatizado
Julho/2026	Validação do funil de custos com a consultoria (payload real de anúncio do WhatsApp p/ auto-match; conferência das fórmulas em produção)	pendente (com Alexis)
Julho/2026	Configurar Coolify + primeiro deploy	pendente (acesso do time)
Ago-Set/2026	Migração dos dados reais (Plano 11: ETL ensaiado + virada única) e reescrita do frontend consumindo a API	planejado (planos escritos)
Agosto/2026	Revitalização UX/UI geral	roadmap do cliente
Setembro/2026	Módulos RH (desenvolvimento humano) e Financeiro	roadmap do cliente
Futuro	Integração Meta Ads (investimento automático) · metas por campanha (descartado por ora) · IA sugerindo planos de ação a partir dos indicadores	backlog

9. Mapa de documentos do projeto

Documento (no repositório trivus-api)	Conteúdo
README.md	visão geral, como rodar, envs, deploy
docs/API_REFERENCE.md	referência completa da API p/ construir o frontend (endpoints, regras, enums)
docs/MIGRACAO_BACKEND.md	engenharia reversa do sistema antigo (domínio completo, 13 seções)
docs/ECOSSISTEMA_TRIVUS.md	spec do ecossistema (empresas/planos/serviços/upsell/cobrança)
docs/db/MODELO_ALVO.md	DDL comentado das 23 tabelas + decisões de modelagem
docs/DEPLOY_COOLIFY.md	passo a passo do deploy (banco, envs, webhook, secrets, troubleshooting)
docs/BILLING_GATEWAY.md	contrato da integração de cobrança e como ativá-la

Documento (no repositório trivus-api)	Conteúdo
docs/plans/00-INDEX.md + 16 planos	planos de implementação executados (baby-steps TDD) + Plano 11 (migração de dados, pendente)

— fim do documento —